TalentScout AI

Hiring Assistant

Technical Documentation

Version **1.0.0**	Repository **Aman-pr/TalentScouting**
Tech Stack **Python · FastAPI · Streamlit · LangChain · Groq**	Live Demo **talent-scouting.vercel.app**

1. Executive Summary

TalentScout AI is an intelligent, AI-powered hiring assistant that streamlines the initial candidate screening process for technical roles. Rather than relying on manual HR overhead for first-round interviews, TalentScout automates profile collection, document analysis, and technical question generation through a natural language conversational interface.

The application is built as a full-stack Python project, with a FastAPI REST backend powering the intelligence layer and a Streamlit-based web frontend providing the recruiter/candidate experience. The system uses Groq-hosted Llama 3.3 70B via LangChain to conduct dynamic, context-aware conversations with candidates.

Key Value Proposition

TalentScout AI eliminates the bottleneck of manual first-round technical screening by automatically generating role-relevant questions based on each candidate's specific technology stack, experience level, and uploaded resume — enabling recruiters to focus only on qualified finalists.

2. Architecture Overview

TalentScout AI follows a classical client-server architecture, where a FastAPI backend handles all AI inference, business logic, and conversation state, while a Streamlit frontend provides the user interface. The two components communicate over a REST API, and a single Docker container packages both together for cloud deployment.

2.1 High-Level Architecture

Frontend Layer	Backend Layer
Streamlit (Python) • Chat UI with session management • Document upload (PDF/DOCX/TXT) • Firebase user authentication • REST calls to backend API	FastAPI (Python) • Conversation state management • LangChain + Groq LLM integration • Resume & JD document parsing • Prompt template orchestration

2.2 Technology Stack

Component	Technology	Purpose
LLM Inference	Groq — Llama 3.3 70B	High-speed language model for conversational AI and question generation
LLM Orchestration	LangChain	Prompt template management and chain orchestration
Backend Framework	FastAPI	High-performance async REST API
Frontend Framework	Streamlit	Reactive web interface and session handling
Authentication	Firebase Auth	Email/password user authentication
Document Parsing	PyMuPDF / python-docx	Resume and JD extraction from PDF/DOCX/TXT
Containerization	Docker + Nginx	Single-container deployment with reverse proxy
Cloud Hosting	Render / Vercel	Backend on Render, frontend on Vercel
Language	Python 3.12+	Primary development language across all layers

3. Project Structure

The repository is organized into a clean monorepo structure with distinct backend and frontend directories, plus a root-level Dockerfile for unified deployment.

```
TalentScouting/
├── backend/                      # FastAPI backend
│   ├── main.py                   # Application entry point & API routes
│   ├── hiring_prompts.py         # LLM prompt templates
│   ├── conversation_manager.py   # Session/state management
│   ├── document_loader.py        # Resume & JD parsing utilities
│   ├── requirements.txt          # Python dependencies
│   └── .env                      # Environment variables (GROQ_API_KEY)
│
├── frontend/                     # Streamlit frontend
│   ├── app.py                    # Main Streamlit application
│   ├── auth.py                   # Firebase authentication logic
│   ├── requirements.txt          # Python dependencies
│   └── *.json                    # Firebase service account credentials
│
├── Dockerfile                    # Unified container definition
├── .gitignore                    # Git exclusions
└── README.md                     # Project overview
```

4. Backend Documentation

The backend is a FastAPI application that serves as the intelligence core of TalentScout AI. It manages LLM-powered conversations, processes candidate profiles, generates technical questions, and parses uploaded documents.

4.1 Entry Point — main.py

The main module initializes the FastAPI application and exposes the core API endpoints. It configures CORS to allow communication from the Streamlit frontend, sets up routing, and delegates business logic to specialized modules.

API Endpoints

Method & Path	Description	Request / Response
POST /chat	Send a user message and receive an AI response.	Request: { session_id, message } | Response: { reply, state }
POST /parse-document	Upload and parse a resume or job description.	Request: multipart/form-data (file) | Response: { extracted_text, parsed_fields }
GET /session/{id}	Retrieve the current state of a conversation session.	Response: { session_id, profile, history }
DELETE /session/{id}	Terminate and clear a conversation session.	Response: { status: 'deleted' }
GET /health	Health check endpoint for deployment probes.	Response: { status: 'ok' }
GET /docs	Swagger UI auto-generated API documentation.	Interactive API documentation browser

4.2 Conversation Manager — conversation_manager.py

This module maintains the state of each candidate's interview session. Since LLMs are stateless, the conversation manager stores the full history and extracted profile data between API calls, ensuring the assistant can provide contextually coherent follow-up responses.

Key Responsibilities

- Maintain per-session conversation history (user + assistant turns)
- Store extracted candidate profile fields across conversation rounds
- Track screening phase progression (greeting → profile → technical questions)
- Provide context injection for each LLM call
- Handle session creation, retrieval, and cleanup

Session State Schema

```
{
  "session_id": "uuid-string",
  "history": [
    { "role": "user", "content": "..." },
    { "role": "assistant", "content": "..." }
  ],
  "profile": {
    "name": "...",
    "email": "...",
    "phone": "...",
    "experience_years": 3,
    "tech_stack": ["Python", "FastAPI", "React"],
    "current_role": "...",
    "location": "..."
  },
  "phase": "technical_questions",
  "questions_asked": 3
}
```

4.3 Prompt Templates — hiring_prompts.py

All LLM interactions are driven by modular, versioned prompt templates defined in this module. Each template is purpose-built for a specific screening phase. The prompts are engineered to extract structured data and generate meaningful technical questions.

Prompt Categories

Prompt Name	Purpose & Design
SYSTEM_PROMPT	Establishes the AI's persona as a professional hiring assistant. Sets tone, language constraints, and behavioral rules to ensure consistent, professional responses throughout the session.
INFO_EXTRACTION_PROMPT	Instructs the LLM to extract structured JSON from candidate responses — covering name, email, phone, years of experience, current role, location, and technology stack.
QUESTION_GENERATION_PROMPT	Given the candidate's declared tech stack and experience level, generates intermediate-to-advanced technical questions tailored specifically to the tools and frameworks they listed.
CONTEXTUAL_RESPONSE_PROMPT	Uses full conversation history and current profile state to generate natural, non-repetitive follow-up messages — preventing the assistant from re-asking for already-provided information.
GREETING_PROMPT	Generates a warm, professional opening message that sets candidate expectations and begins the profile collection flow.

Prompt Design Principles

- Strict JSON mode enforced for extraction prompts to enable reliable parsing
- Markdown stripping wrapper applied to all LLM outputs before JSON deserialization
- Conversation history injected as context to prevent repetitive questioning
- Tech stack is extracted as an array to enable per-technology question generation
- Difficulty calibration based on stated years of experience (junior vs. senior)

4.4 Document Loader — document_loader.py

This module handles all file ingestion logic. It provides a unified interface for reading content from three file formats — PDF, DOCX, and plain text — and passes the extracted text to the LLM for structured analysis.

File Format	Parsing Strategy
PDF (.pdf)	Uses PyMuPDF (fitz) to extract text page-by-page, handling multi-column layouts and embedded text objects.
Word Document (.docx)	Leverages python-docx to iterate through document paragraphs and table cells, preserving structural context.
Plain Text (.txt)	Direct file read with UTF-8 encoding; no preprocessing required.

5. Frontend Documentation

The frontend is a Streamlit web application that provides the primary user interface for TalentScout AI. It manages user sessions, renders the conversational chat UI, handles file uploads, and communicates with the backend over REST.

5.1 Main Application — app.py

The app.py module is the single entry point for the Streamlit frontend. It orchestrates all UI components, manages application state via Streamlit's session state, and handles the send/receive flow with the backend API.

Core UI Flows

- Authentication Gate: If user is not logged in, display the login/signup form (handled by auth.py). On successful authentication, initialize a new conversation session.
- Chat Interface: Render conversation history in a scrollable chat window with distinct user and assistant message bubbles. Handle text input and send messages to the backend /chat endpoint.
- Document Upload: Provide a sidebar file uploader for resumes or job descriptions. On upload, POST the file to /parse-document and inject the extracted text as context.
- Session Control: Allow users to reset or terminate their session, clearing all profile data and history.
- Profile Summary Panel: Display a real-time summary of extracted candidate profile fields as they are collected during the conversation.

5.2 Authentication — auth.py

Authentication is handled via Firebase Authentication using the Email/Password provider. This module wraps the Firebase REST API for sign-in, sign-up, and token validation, allowing the Streamlit app to enforce access control.

Authentication Flow

1. User submits email and password through the Streamlit login form.
2. auth.py calls the Firebase Authentication REST API (/identitytoolkit/v3/relyingparty/verifyPassword) with the provided credentials.
3. On success, Firebase returns an ID token that is stored in Streamlit session state.
4. The ID token is validated server-side using the Firebase Admin SDK on protected operations.
5. On logout or token expiry, session state is cleared and the user is redirected to the login screen.

6. AI & Prompt Engineering

6.1 Model Selection

TalentScout AI uses the Llama 3.3 70B Versatile model hosted on Groq's inference infrastructure. This model was selected for the following reasons:

Selection Criterion	Detail
Speed / Latency	Groq's LPU hardware delivers near-instant token generation, essential for real-time conversational UX where delays break the interview flow.
Reasoning Quality	Llama 3.3 70B provides strong instruction-following and structured output (JSON) generation, which is critical for reliable profile extraction.
Context Window	The 128K token context window allows the full conversation history and uploaded documents to be passed in a single inference call.
Cost Efficiency	Groq's pricing provides a compelling cost-per-token ratio for high-throughput screening workloads.
Open Weights	The open-weight nature of Llama models provides flexibility for future fine-tuning or self-hosting if required.

6.2 Information Extraction Approach

The most technically challenging aspect of the system is reliably extracting structured candidate profiles from unstructured natural language responses. The extraction pipeline works as follows:

6. The INFO_EXTRACTION_PROMPT instructs the LLM to return only a JSON object — no preamble, no explanation, no markdown fences.
7. After the LLM call returns, a post-processing wrapper strips any residual markdown (e.g., ```json markers) before passing to json.loads().
8. Extracted fields are merged into the existing session profile, so partial updates from each turn accumulate into a complete profile over time.
9. The system gracefully handles missing fields — the LLM is instructed to return null for any field it cannot determine from the candidate's response.

6.3 Technical Question Generation

Once the candidate's tech stack is collected, TalentScout AI generates tailored technical questions using the QUESTION_GENERATION_PROMPT. The prompt instructs the model to:

- Generate questions specifically for each technology listed by the candidate (not generic questions)
- Calibrate difficulty to intermediate-to-advanced level, adjusted for the candidate's years of experience
- Avoid repeating topics already covered in previous questions in the session
- Produce questions that test practical, applied knowledge rather than pure trivia
- Return questions in a structured format that the frontend can render clearly

7. Installation & Setup

7.1 Prerequisites

Requirement	Details
Python	Version 3.12 or higher. Earlier versions may work but are not tested.
Groq API Key	Obtain from console.groq.com. The free tier is sufficient for development.
Firebase Project	Required for authentication. Free Spark plan is sufficient.
Git	For cloning the repository.
Docker (optional)	Required only for containerized deployment.

7.2 Local Development Setup

Step 1 — Clone the Repository

```
git clone https://github.com/Aman-pr/TalentScouting.git
cd TalentScouting
```

Step 2 — Create and Activate a Virtual Environment

```
python -m venv venv
# On macOS/Linux:
source venv/bin/activate
# On Windows:
venv\Scripts\activate
```

Step 3 — Install Dependencies

```
pip install -r backend/requirements.txt
pip install -r frontend/requirements.txt
```

Step 4 — Configure Environment Variables

Create a .env file inside the backend/ directory:

```
# backend/.env
GROQ_API_KEY=your_groq_api_key_here
```

Step 5 — Configure Firebase

10. Go to console.firebase.google.com and create or select a project.

11. In the Build menu, open Authentication and enable the Email/Password provider.
12. Navigate to Project Settings > General and copy the Web API Key.
13. Open frontend/auth.py and paste the key as the FIREBASE_API_KEY value.
14. Go to Project Settings > Service Accounts and generate a new private key (JSON).
15. Save the downloaded JSON file into the frontend/ directory.
16. Update the _CRED_PATH variable in frontend/auth.py to match your JSON filename.

7.3 Running the Application

Start the Backend (Terminal 1)

```
cd backend
python main.py
# Backend available at: http://localhost:8000
# Swagger docs at:      http://localhost:8000/docs
```

Start the Frontend (Terminal 2)

```
cd frontend
streamlit run app.py
# Frontend available at: http://localhost:8501
```

8. Docker Deployment

The project ships a Dockerfile that packages both the backend and frontend into a single unified container. An internal Nginx reverse proxy routes traffic between the two services, enabling single-port deployment on cloud platforms.

8.1 Container Architecture

Single-Container Design

Rather than requiring Docker Compose with multiple containers, TalentScout AI runs both FastAPI (port 8000) and Streamlit (port 8501) inside a single container. Nginx listens on the exposed port and routes requests to the appropriate internal service based on URL path.

8.2 Local Docker Usage

Build the Image

```
docker build -t talentscout-app .
```

Run the Container

```
docker run -p 8000:8000 \
  -e GROQ_API_KEY=your_key_here \
  talentscout-app
```

Development Mode (with Live Reload)

```
docker run -p 8501:8501 -p 8000:8000 \
  -v $(pwd):/app \
  talentscout-app
```

8.3 Publishing to Docker Hub

```
docker login
docker tag talentscout-app yourusername/talentscout-app:latest
docker push yourusername/talentscout-app:latest

# Others can then run with:
docker run -p 8501:8501 -p 8000:8000 \
  -e GROQ_API_KEY=your_key \
  yourusername/talentscout-app:latest
```

8.4 Cloud Deployment (Render / Railway / Fly.io)

17. Push the repository to GitHub.
18. On Render (or your chosen platform), create a new Web Service and select Docker as the runtime.
19. Set the following environment variables in the platform dashboard:

Variable	Value
`GROQ_API_KEY`	Your Groq API key from console.groq.com
`ENVIRONMENT`	production — enables correct API routing via Nginx

20. Deploy — the platform will build the Docker image and bind it to the dynamically assigned $PORT.

Important Note

Ensure the Firebase Service Account JSON file is present in the frontend/ directory before pushing to GitHub, or use the platform's secret file injection feature to provide it at runtime without committing credentials to source control.

9. Configuration Reference

9.1 Environment Variables

Variable	Location	Description
GROQ_API_KEY	backend/.env	Required. API key for Groq inference. Obtain from console.groq.com.
FIREBASE_API_KEY	frontend/auth.py	Required. Firebase Web API Key from Project Settings > General.
ENVIRONMENT	Docker / Cloud	Set to 'production' to enable Nginx-based routing in deployed containers.
_CRED_PATH	frontend/auth.py	Path to Firebase service account JSON file, relative to the frontend/ directory.

9.2 LLM Configuration

Parameter	Value & Notes
Model ID	llama-3.3-70b-versatile — the model string used in all Groq API calls.
Max Tokens	Configurable per prompt type; extraction prompts use lower limits, question generation uses higher.
Temperature	Low values (~0.3) for extraction prompts for consistency; moderate values (~0.7) for conversational responses.
Top-P	Default Groq values used; can be tuned in hiring_prompts.py for more diverse question output.
Context Strategy	Full conversation history is injected on every call; no summarization or truncation unless history exceeds context window.

10. Engineering Challenges & Solutions

10.1 Consistent JSON Parsing from LLMs

Challenge	Solution
LLM responses would sometimes include markdown formatting (```json ... ```) or explanatory preamble before the JSON object, causing json.loads() to throw exceptions and breaking the extraction pipeline.	Implemented a robust post-processing wrapper that strips markdown code fences (```json, ```) and trims whitespace before passing the string to the JSON parser. Also updated prompts to explicitly forbid preamble and markdown in extraction responses.

10.2 Virtual Environment Path Corruption

Challenge	Solution
Renaming the project directory caused hardcoded absolute paths in the virtual environment's bin/ folder to break, making Streamlit and other installed tools unresolvable — resulting in 'command not found' errors.	Implemented a recursive search-and-replace script targeting the venv/bin/ directory to update all internal path references from the old directory name to the new one — restoring environment integrity without needing a full reinstall.

10.3 Stateless LLM Session Management

Challenge	Solution
LLMs are inherently stateless — each API call is independent. Without explicit state management, the assistant would lose context of what had already been asked or what candidate information had already been provided.	The conversation_manager.py module implements server-side session storage that maintains full conversation history, the accumulated candidate profile, and the current screening phase. This complete state is injected into every LLM prompt call.

11. Code Quality & Best Practices

11.1 Design Principles

Principle	Implementation
Modularity	Business logic is strictly separated into dedicated modules: conversation management, prompt templates, and document loading. Each module has a single, well-defined responsibility.
Type Safety	Python type hints are used throughout the codebase. FastAPI leverages Pydantic models for request and response validation, providing automatic schema enforcement.
Error Handling	All API endpoint handlers are wrapped in comprehensive try-except blocks. Error responses include descriptive messages and appropriate HTTP status codes (400, 422, 500).
Minimal Comments	The codebase follows a self-documenting approach where function names, type hints, and module structure communicate intent — comments are reserved for non-obvious logic.
Clean Prompts	Prompt templates are stored as named constants, not inline strings. This allows prompts to be versioned, tested, and updated independently of business logic.
Environment Separation	Secrets and configuration are externalized to .env files and environment variables, never hardcoded in source files.

11.2 API Design

- RESTful conventions followed for all endpoints (correct HTTP verbs, status codes, and resource-based URLs)
- Request and response models defined with Pydantic for automatic validation and OpenAPI schema generation
- CORS configured explicitly to allow only the expected frontend origin in production
- Auto-generated Swagger documentation available at /docs for easy API exploration and testing

12. Live Deployments & Links

Resource	URL
Application UI (Render)	https://talentscouting.onrender.com
Backend API (Vercel)	https://talent-scouting.vercel.app/
API Documentation (Swagger)	https://talent-scouting.vercel.app/docs
GitHub Repository	https://github.com/Aman-pr/TalentScouting

13. Potential Future Improvements

Area	Proposed Enhancement
Persistent Storage	Replace in-memory session state with a database (e.g., PostgreSQL or Redis) to persist sessions across server restarts and support multi-instance horizontal scaling.
Candidate Scoring	Implement an automated scoring engine that evaluates candidate responses to technical questions and generates a structured scorecard for recruiters.
Multi-language Support	Extend prompts to support non-English candidates by detecting input language and responding in kind.
Fine-tuned Model	Fine-tune a smaller model (e.g., Llama 3 8B) on domain-specific hiring conversations to reduce latency and cost while maintaining quality.
ATS Integration	Add webhook or API integrations with popular Applicant Tracking Systems (Greenhouse, Lever, Workday) to automatically push screened candidate profiles.
Async Question Evaluation	Use a secondary LLM call to evaluate candidate answers to technical questions and flag strong or weak responses for recruiter review.
Voice Interface	Add speech-to-text input and text-to-speech output to support phone-screen-style audio interviews.
Analytics Dashboard	Build a recruiter-facing analytics dashboard showing screening funnel metrics, common tech stacks, and time-to-screen statistics.

Appendix: Quick Reference

A.1 Common Commands

```
# Clone repository
git clone https://github.com/Aman-pr/TalentScouting.git

# Install all dependencies
pip install -r backend/requirements.txt
pip install -r frontend/requirements.txt

# Run backend
cd backend && python main.py

# Run frontend
cd frontend && streamlit run app.py

# Build Docker image
docker build -t talentscout-app .

# Run Docker container
docker run -p 8000:8000 -e GROQ_API_KEY=your_key talentscout-app

# Health check
curl http://localhost:8000/health
```

A.2 Dependency Summary

Package	Layer	Purpose
fastapi	Backend	REST API framework
uvicorn	Backend	ASGI server for FastAPI
langchain	Backend	LLM orchestration and prompt management
langchain-groq	Backend	Groq provider integration for LangChain
python-dotenv	Backend	Environment variable loading from .env files
PyMuPDF (fitz)	Backend	PDF text extraction
python-docx	Backend	DOCX document parsing
pydantic	Backend	Request/response data validation
streamlit	Frontend	Web UI framework
requests	Frontend	HTTP calls from frontend to backend API
firebase-admin	Frontend	Firebase server-side token validation

TalentScout AI — Technical Documentation

Generated from repository: github.com/Aman-pr/TalentScouting